TED University

SENG 491 – Senior Project

SpecOps: Specification-to-Code Pattern Enforcer

Fall 2025

Team Members:

Shahdad Peykani – 99111760026 – Software Engineering

Muhammad Ozar Mirza – 99687965054 – Software Engineering

Abdirahman Tahlil Farah Hussein – 99717977422 – Software Engineering

Supervisor: Prof. Tansel Dökeroğlu

Jury Members:

Prof. Emin Kuğu

Prof. Elif Kurtaran Özbudak

Contents

1. Introduction 4
1.1. Purpose 4
1.2. Product Scope 4
1.3. Description 4
2. Overall Description 4
2.1. Product Perspective 4
2.2. Product Functions 4
2.3. Operating Environment 5
2.4. Constraints 5
2.5. Professional and Ethical Issues 5
3. External Interface Requirements 6
3.1. User Interface 6
3.2. Hardware Interfaces 6
3.3. Software Interfaces 6
4. System Features 6
4.1. SRS Generation & Pattern Selection 6
4.1.1. Description and Priority 6
4.1.2. Stimulus/Response Sequences 6
4.1.3. Functional Requirements 7
4.2. Pattern-Compliant Code Generation 7
4.2.1. Description and Priority 7
4.2.2. Stimulus/Response Sequences 7
4.3. Automated Testing & Quality Gating 7
4.3.2. Stimulus/Response Sequences 7
4.3.3. Functional Requirements 7
4.4. Project Asset Generation & Web Interface 8
4.4.1. Description and Priority 8
4.4.2. Stimulus/Response Sequences 8
4.4.3. Functional Requirements 8
5. Other Nonfunctional Requirements 8
5.1. Performance Requirements 8
5.2. Safety Requirements 8
5.3. Security Requirements 9
5.4. Usability Requirements 9

5.5. Accuracy Requirements 9
6. Appendices 9
6.1. Appendix A: Glossary 9
6.2. Appendix B: Analysis Models 10
6.3. Appendix C: List to Be Determined (TBD) 11
7. References 11

1. Introduction

1.1. Purpose

This document describes the Software Requirements Specification (SRS) for SpecOps, an AI-powered framework designed to enforce software engineering discipline from the initial user prompt. It is intended for stakeholders, including the development team, project supervisors, jury members, and potential users, to serve as a definitive guide to the system's capabilities, constraints, and high-level design.

1.2. Product Scope

SpecOps addresses the problem of "vibe coding" and architectural debt introduced by the use of LLMs in software development. It is a structured pipeline that transforms a natural language prompt into a formal SRS, a pattern-compliant codebase, and associated project assets. Unlike the generic code generation, SpecOps acts just like a software engineer by using the best software practices, design patterns, and quality from the very beginning. The product is targeted at developers and teams who seek to accelerate development while maintaining good standards of code quality, security, and maintainability.

1.3. Description

The rapid adoption of Large Language Models (LLMs) in software development has led to a "vibe coding" problem, where quickly prompting for code results in significant architectural debt, inconsistent quality, and security gaps. Teams waste their time in refactoring and cleaning outputs generated by AI, and collaboration suffers the consequences by being without a shared and disciplined structure.

SpecOps is an AI-powered framework designed to replace this ad-hoc approach with a structured, disciplined software engineering pipeline. It transforms a plain natural-language prompt into a formal Software Requirements Specification (SRS), then produces pattern-compliant, fully-tested code and organized project assets. By enforcing proven design patterns and quality gates from the outset, SpecOps ensures that quality, security, and maintainability are built-in, not an afterthought.

2. Overall Description

2.1. Product Perspective

SpecOps is envisioned as a web application that oversees multiple AI agents and external development tools. It sits between the developer's needs and the final completed code, acting as an intelligent middleman that ensures engineering practices.

2.2. Product Functions

The core functions of SpecOps are:

- Automatically generate a structured SRS from a natural language prompt.
- Select and justify appropriate software design patterns.
- Generate a complete, pattern-compliant codebase.
- Automatically generate and run unit and integration tests.
- Perform static code analysis and security scanning.
- Generate project documentation, setup scripts, and CI/CD configurations.

- Provide a web-based interface for user interaction.

2.3. Operating Environment

Software: The system will be deployed as a web application, accessible via modern browsers. The backend will be built with Python. It will interact with LLM APIs (e.g., Gemini), code linters (e.g., Pylint), and security scanners (e.g., Bandit).

Hardware: The system will rely on cloud-based APIs for these services.

2.4. Constraints

AI Model Constraints: The quality and consistency of SpecOps' output are dependent on the capabilities and limitations of the underlying LLM (e.g., Gemini 2.5 pro). Model reasoning errors or token limits may occasionally affect the depth of analysis.

Pattern Library Constraints: The framework's effectiveness is bounded by the width and depth of its internal pattern library. It cannot enforce patterns it does not know, and pattern selection may not always be optimal for niche requirements.

Toolchain Integration Constraints: The performance and reliability of code analysis and security scanning are tied to the integrated open-source tools. Their rules and capabilities may not cover every possible case.

Language and Framework Constraints: The initial version of SpecOps will be constrained to specific programming languages and frameworks (e.g., Python). Expanding to other ecosystems (e.g., JavaScript, Java) requires significant additional development.

2.5. Professional and Ethical Issues

Intellectual Property & Attribution: SpecOps generates code based on publicly available patterns and best practices. We must ensure it does not reproduce significant, copyrighted code from its training data without changes.

Reliability and Accountability: Code generated by SpecOps is intended for assistance, not autonomous deployment. There is a risk that users might over-rely on its output. Clear disclaimers are necessary, and the framework must be transparent about its reasoning to allow for human oversight.

Security Responsibility: While SpecOps integrates security scanners, it cannot guarantee completely vulnerability-free code. The ultimate responsibility for security review lies with the human developer using the tool.

Bias in Pattern Selection: The RAG system or the underlying LLM could develop a bias towards certain patterns, potentially overlooking better alternatives. The system must be designed for explainability to mitigate this.

3. External Interface Requirements

3.1. User Interface

The system shall provide a single-page web application (SPA) with a clean, intuitive interface. The main components will include:

A text area for entering the project prompt.

A real-time preview pane for the generated SRS.

A file-tree view of the generated code structure.

An "Explain Mode" panel that details pattern justifications.

Tabs for viewing test reports, security scan results, and generated documentation.

3.2. Hardware Interfaces

There are no direct hardware interfaces. The system runs on standard server hardware.

3.3. Software Interfaces

LLM API (e.g., Gemini 2.5 Pro): For SRS generation, pattern selection, and code generation.

Static Analysis Tools (e.g., Pylint): For checking the quality if the code.

Security Scanners (e.g., Bandit): For identifying security vulnerabilities in the generated Python code.

Testing Framework (e.g., Pytest): For executing generated tests and calculating coverage.

Version Control System (e.g., Git): For initializing the output project as a repository.

4. System Features

This section details the high-priority features of the SpecOps system.

4.1. SRS Generation & Pattern Selection

4.1.1. Description and Priority

High Priority. This is the foundational feature. The system must accurately interpret a user's natural language description and formalize it into a structured SRS, then select the most appropriate architectural and design patterns.

4.1.2. Stimulus/Response Sequences

- Stimulus: User submits a natural language prompt (e.g., "Create a REST API for a blog with users, posts, and comments").
- Response: The system displays a well-structured SRS document with sections for Scope, Actors, Functional Requirements, and Non-Functional Requirements.
- Stimulus: The SRS is generated.
- Response: The system displays a list of selected patterns (e.g., MVC, Repository) with clear justifications linked to the SRS requirements.

4.1.3. Functional Requirements

- FR-1: The system shall automatically generate a structured SRS from a natural language user prompt.
- FR-2: The system shall select and justify appropriate software design patterns based on the generated SRS.

4.2. Pattern-Compliant Code Generation

4.2.1. Description and Priority

High Priority. This feature translates the SRS and selected patterns into a working, well-organized codebase.

4.2.2. Stimulus/Response Sequences

- Stimulus: User confirms the generated SRS and pattern selection.
- Response: The system generates a complete codebase in a standard directory structure, implementing the chosen patterns.

4.2.3. Functional Requirements

- FR-3: The system shall generate a complete, pattern-compliant codebase for the specified project.

4.3. Automated Testing & Quality Gating

4.3.1. Description and Priority

High Priority. This feature ensures the generated code is reliable and meets quality standards.

4.3.2. Stimulus/Response Sequences

- Stimulus: Code generation is complete.
- Response: The system automatically generates and runs unit/integration tests, then displays a test coverage report.
- Stimulus: Tests are run.
- Response: The system runs static analysis and security scanners, displays a report, and automatically attempts to fix common, low-severity issues.

4.3.3. Functional Requirements

- FR-4: The system shall automatically generate unit and integration tests for the generated code.

- FR-5: The system shall integrate static analysis and security scanners, attempting to fix common issues and reporting critical findings.

4.4. Project Asset Generation & Web Interface

4.4.1. Description and Priority

Medium Priority. This feature packages the final output and provides the user interface.

4.4.2. Stimulus/Response Sequences

- Stimulus: All generation and analysis steps are complete.
- Response: The system generates a README, setup scripts, and a basic CI/CD config file, and makes the entire project available for download.
- Stimulus: User navigates to the application URL.
- Response: The web interface loads, allowing the user to input a prompt and view all generated artifacts.

4.4.3. Functional Requirements

- FR-6: The system shall automatically generate project documentation, setup scripts, and a basic CI/CD configuration.
- FR-7: The system shall provide a web-based editor for users to input prompts and view results.

5. Other Nonfunctional Requirements

5.1. Performance Requirements

The system should process a standard project prompt and generate the initial SRS and high-level design within 5-10 seconds.

Full code generation, testing, and analysis for a medium-complexity project should be completed within 2-5 minutes.

5.2. Safety Requirements

Not applicable in the traditional sense. "Safety" in this context relates to the reliability of the output. The system must include graceful error handling and clear user feedback if a sub-process (e.g., code generation, security scan) fails.

5.3. Security Requirements

- The system must securely handle and never log API keys or other sensitive credentials used to access LLM services.
- User prompts and generated code shall not be stored persistently without explicit user permission.
- All generated code must pass integrated security scans with 0 critical vulnerabilities before being presented as final.
- The web interface shall be protected against common web vulnerabilities (e.g., XSS, CSRF).

5.4. Usability Requirements

- The web interface should be intuitive, allowing developers to use the system with minimal learning curve.
- The "Explain Mode" must provide clear, understandable justifications for pattern choices.

5.5. Accuracy Requirements

- Pattern enforcement should be ≥ 85% accurate against a validated benchmark.

- Generated code should have a test coverage of ≥ 80%.

6. Appendices

6.1. Appendix A: Glossary

I. SRS (Software Requirements Specification): A formal document describing the features and constraints of a software project.
II. LLM (Large Language Model): A type of AI model trained on vast amounts of text data to understand and generate human-like language.
III. RAG (Retrieval-Augmented Generation): A technique that improves AI response quality by retrieving information from a knowledge base before generating an answer.
IV. Design Pattern: A general, reusable solution to a commonly occurring problem in software design.
V. CI/CD (Continuous Integration/Continuous Deployment): A method to frequently deliver apps to customers by introducing automation into the stages of app development.

6.2. Appendix B: Analysis Models

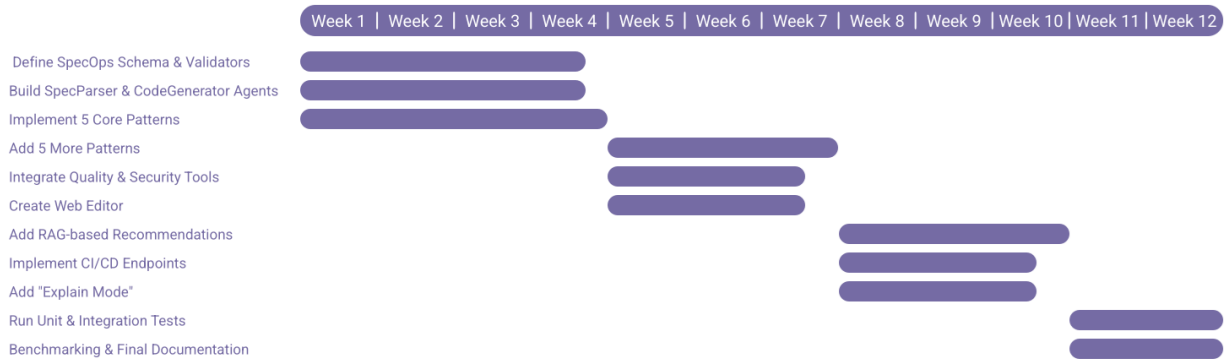

Figure 1: Proposed Gantt Chart

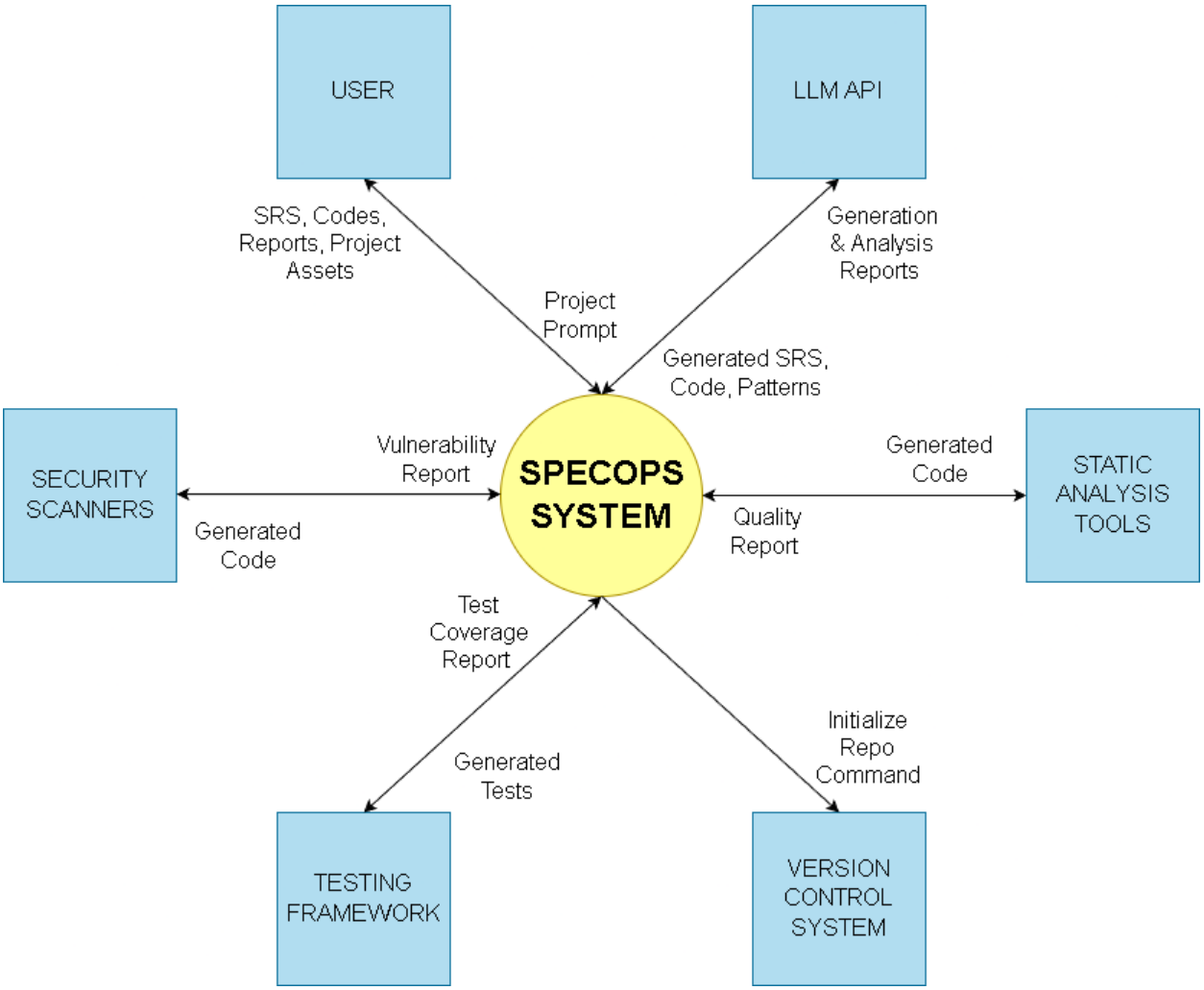

Figure 2: High-Level System Architecture

6.3. Appendix C: List to Be Determined (TBD)

I. Specific criteria for "Pattern Compliance" accuracy measurement.
II. The exact set of 10+ patterns to be implemented in the final library.
III. User authentication and project history management for the web editor.
IV. Support for project templates beyond basic web APIs (e.g., data science projects, mobile apps).

7. References

[1] ACM, “ACM Code of Ethics and Professional Conduct,” 2023. [Online]. Available: https://www.acm.org/code-of-ethics

[2] IEEE, “IEEE Code of Ethics,” n.d. [Online]. Available: https://www.ieee.org/about/corporate/governance/p7-8.html

[3] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[4] OWASP Foundation, “OWASP Top Ten,” 2023. [Online]. Available: https://owasp.org/www-project-top-ten/

[5] System Architecture Diagram, “Created with draw.io,” Accessed: Nov. 08, 2025. [Online]. Available: https://app.diagrams.net/

[6] Gantt Chart, “Created with app.gantt.io,” Accessed: Nov. 08, 2025. [Online]. Available: https://app.gantt.io/